

Agenda

- ① What is a recursion?
- ② Recursion Tree | Tracing code
- ③ How to approach recursive solution?
- ④ Few examples

What is a Recursion?

- Function calling itself is called recursion.
- A recursive method solves a problem by calling a copy of itself to work on a smaller problem.
- It is important to ensure that the recursion terminates.

- Each time the function call itself with a slightly simpler version of the original problem.
- Recursive code is generally shorter and easier to write than iterative code.
- Solution to some problems are easier to formulate recursively.

Recursion Tree

```
int main()
```

```
{  
    int k;  
    k = f1(3);  
    printf("%d", k);  
}
```

```
int f1(int n)
```

```
{  
    int s;  
    if(n==1)  
        return 1;  
    s = n + f1(n-1);  
    return(s);  
}
```

```
int f1(int n)
```

```
n | 2 | if(n==1)  
    |   | return 1;  
s | 3 | s = n + f1(n-1);  
    |   | return(s);
```

```
int f1(int n)
```

```
n | 3 | if(n==1)  
    |   | return 1;  
s | 6 | s = n + f1(n-1);  
    |   | return(s);
```

```
main()
```

```
k | 6 |  
    |   | k = f1(3);  
    |   | printf("%d", k);
```

```
int f1(int n)
```

```
n | 1 | if(n==1)  
    |   | return 1;  
s |   | s = n + f1(n-1);  
    |   | return(s);
```

Sum of first n
natural numbers

How to approach a Recursive Problem?

Write a recursive function to calculate sum of first n natural numbers.

```
int Sum(int n)
```

```
{
```

```
    if (n == 1)
```

```
        return 1;
```

```
    return n + Sum(n-1);
```

① $Sum(n) = 1 + 2 + 3 + 4 + \dots + n$

RC ② $n + Sum(n-1) = 1 + 2 + 3 + \dots + n-1$

BC ③

$n \geq 1$ recursion

$n == 0$ recursion stop

$n == 1$ return 1

```
}
```

$$Sum(n) = 1 + 2 + 3 + 4 + \dots + n-1 + n$$

$$Sum(n-1) = \underline{1 + 2 + 3 + 4 + \dots + n-1}$$

15 sum(5);

¹⁰
5 + sum(4)

⁶
4 + sum(3)

³
3 + sum(2)

¹
2 + sum(1)

¹
↑
1

return 1

1

→

n == 1

return n + sum(n-1) → otherwise

Write a recursive function to calculate factorial of n .

```
int fact(int n)
```

```
{
```

```
    if (n == 0 || n == 1)
```

```
        return 1;
```

```
    return n * fact(n-1);
```

```
}
```

①

RC ②

BC ③

$\text{fact}(n) = 1 \times 2 \times 3 \times \dots \times n$

$n \times \text{fact}(n-1) = 1 \times 2 \times 3 \times \dots \times (n-1)$

$0! \text{ or } 1! = 1$

$n! = n \times (n-1)!$

Write a recursive function to print first n natural numbers.

```
void printN(int n)
{
    if (n > 0)
    {
        printN(n-1);
        printf("%d", n);
    }
}
```

① printN(n) 1 2 3 4 n

RC ② [- printN(n-1) 1 2 3 4 ... n-1

BC ③ [- printf("%d", n);

{ n == 0 BC

{ n > 0 RC